Comparative Evaluation of Simulated Annealing, Tabu Search, and Particle Swarm Optimization Across Practical Optimization Problems

Juan Miguel Miranda
juan_miranda@dlsu.edu.ph

Lance Xavier Lim
lance_xavier_lim@dlsu.edu.ph

Julian Johan Briones
Julian_briones@dlsu.edu.ph

ABSTRACT

This study compares Simulated Annealing, Tabu Search, and Particle Swarm Optimization across route optimization, job scheduling, and machine-learning feature selection. The review looks at their reported performance, efficiency, scalability, parameter sensitivity, and implementation requirements. When feasible, small sample tests will also be used to compare the behavior of the algorithms with patterns found in previous studies. The goal is not to identify one universal winner, but to develop a framework showing where each algorithm may be more suitable.

Categories and Subject Descriptors
G.1.6 [Numerical Analysis]: Optimization - global optimization, stochastic programming; I.2.8 [Artificial Intelligence]: Problem Solving, Control Methods, and Search - heuristic methods.

General Terms
Algorithms, Performance, Experimentation, Measurement.

Keywords
Simulated Annealing, Tabu Search, Particle Swarm Optimization, route optimization, job scheduling, feature selection, metaheuristics.

1. INTRODUCTION

Optimization problems involve finding a good solution from many possible alternatives. In route planning, job scheduling, and machine-learning feature selection, the number of possible solutions can grow rapidly enough that exhaustive search becomes impractical. Metaheuristic methods address this difficulty by guiding the search toward promising regions without examining every candidate. This study focuses on Simulated Annealing, Tabu Search, and Particle Swarm Optimization because they search for solutions in different ways and may perform differently depending on the type of problem.

1.1 Main Research Question

How do Simulated Annealing, Tabu Search, and Particle Swarm Optimization differ in their effectiveness across route optimization, job scheduling, and machine-learning feature selection?

1.2 Supporting Research Questions

1. How do Simulated Annealing, Tabu Search, and Particle Swarm Optimization compare in route optimization, particularly in terms of route quality, convergence behavior, computational efficiency, and scalability?
2. How do Simulated Annealing, Tabu Search, and Particle Swarm Optimization compare in job scheduling in terms of schedule quality, makespan, computational efficiency, convergence behavior, and scalability?
3. How do Simulated Annealing, Tabu Search, and Particle Swarm Optimization compare in machine-learning feature selection in terms of predictive performance, feature reduction, computational cost, and result stability?
4. What implementation requirements, parameter sensitivities, strengths, and limitations affect the suitability of each algorithm for the three application areas?
5. To what extent do the results of the small sample tests support or differ from the patterns identified in the reviewed literature?

1.3 Study Scope and Deliverable

This study will primarily conduct a comparative review of existing research on Simulated Annealing, Tabu Search, and Particle Swarm Optimization across route optimization, job scheduling, and machine-learning feature selection. The comparison will consider reported solution quality, convergence behavior, computational efficiency, scalability, parameter sensitivity, and implementation requirements. The final criteria may be adjusted depending on what information is consistently available across the reviewed studies.

To supplement the literature-based comparison, the researchers will conduct small sample tests representing each application area when feasible. These tests will provide practical demonstrations under simplified and controlled conditions. Their results will be compared with patterns found in the literature, but they will not be treated as proof that one algorithm is always better than the others.

The final output will be a Comparative Evaluation Framework that identifies the reported strengths, limitations, implementation requirements, and suitable application conditions of the three algorithms. The framework will combine the literature synthesis with observations from the sample tests to provide practical guidance on which algorithm may be more suitable for particular problem structures and priorities.

2. REVIEW OF RELATED LITERATURE

2.1 Metaheuristic Optimization

Optimization refers to the process of identifying the best feasible solution from a set of alternatives subject to defined objectives and constraints. Many real-world problems, including route planning, job scheduling, and machine-learning feature selection, become increasingly difficult as the number of possible

solutions grows. Although exact algorithms can guarantee optimality, their computational requirements may become impractical for large and complex problem instances. Researchers therefore commonly use heuristic and metaheuristic methods to obtain high-quality solutions within reasonable computational limits.

Heuristics are methods designed for specific problems, while metaheuristics are more general search methods that can be adapted to different problems. They do not guarantee the best possible answer, but they try to find good solutions without checking every possibility. Their usefulness is especially apparent in combinatorial problems, where the number of feasible configurations may increase exponentially or factorially as the problem becomes larger. A central concern in metaheuristic design is the balance between exploration and exploitation.

Exploration involves examining new or less-visited regions of the solution space, while exploitation involves refining solutions in areas already known to be promising. Too much exploitation can trap the algorithm in a local optimum, while too much exploration can waste time checking weak solutions. Effective metaheuristics try to balance both. (Youssef et al., 2001).

Metaheuristic algorithms may generally be classified as either single-solution or population-based methods. Single-solution methods maintain one current candidate and repeatedly move to a neighboring solution. Simulated Annealing and Tabu Search belong to this category, although they use substantially different mechanisms for escaping local optima. Simulated Annealing relies on temperature-dependent probabilistic acceptance, whereas Tabu Search uses adaptive memory and temporary restrictions on recently performed moves. Population-based methods maintain several candidate solutions simultaneously. Particle Swarm Optimization is a population-based method in which particles exchange information through personal and collective experience. This population structure allows PSO to examine several areas of the solution space during the same search. However, rapid information sharing may also cause the particles to converge prematurely when the population follows an inferior leader too quickly (Kennedy & Eberhart, 1995; Sengupta et al., 2019).

The No Free Lunch principle indicates that no optimization algorithm can be universally superior across every possible problem. An algorithm that performs strongly for one problem category may perform poorly for another. An algorithm's performance depends on the problem, how solutions are represented, its parameter settings, and the available computational budget. Results from one application should therefore not be treated as proof that the same algorithm will perform best everywhere. Simulated Annealing, Tabu Search, and Particle Swarm Optimization were selected for the present study because they represent three substantially different search strategies. Simulated Annealing represents probabilistic single solution search, Tabu Search represents memory-guided neighborhood search, and Particle Swarm Optimization represents cooperative population-based search. Comparing them across several problem types may provide a clearer understanding of the conditions under which each method performs effectively.

2.2 Simulated Annealing

Simulated Annealing is a single-solution metaheuristic introduced by Kirkpatrick et al. (1983). It is inspired by the process of heating and slowly cooling material. In optimization, temperature controls how willing the algorithm is to accept worse solutions while searching for a better one. The algorithm begins with an initial candidate solution and generates a neighboring solution through a problem-specific modification. If the neighbor improves the objective value, it is accepted immediately. A worse neighbor may also be accepted according to a probability determined by the size of the deterioration and the current temperature. A common expression for this rule is Acceptance probability = exp(-cost increase / temperature). When temperature is high, even some substantially worse solutions may be accepted. This helps the algorithm escape local optima. As the temperature decreases, it becomes less willing to accept worse solutions and focuses more on improving the current result. Unlike ordinary hill climbing, which accepts only better moves, Simulated Annealing may temporarily accept worse ones. This can help it escape a local optimum and reach a better solution later.

The cooling schedule is one of the most important parts of Simulated Annealing. It controls how quickly the temperature decreases and therefore how quickly the algorithm shifts from exploration to refinement. A common geometric schedule uses New temperature = cooling factor × current temperature. A cooling factor close to one produces slower cooling and prolonged exploration, while a smaller factor causes more rapid cooling and earlier exploitation. Cooling too quickly may cause premature convergence because the algorithm loses its ability to escape local optima before enough of the solution space has been explored. Cooling too slowly may improve exploration but can require excessive computation. The appropriate balance depends on the structure of the problem, the cost scale, and the neighborhood operator. Youssef et al. (2001) demonstrated that the numerical scale of the objective function can significantly affect Simulated Annealing. In their VLSI floorplanning experiment, Youssef et al. (2001) found that the scale of the objective values caused SA to accept too many poor moves. After adjusting the scale, its performance improved substantially. Their findings show that temperature values cannot be selected independently of the objective-function scale.

An apparently poor Simulated Annealing result may therefore reflect inadequate calibration rather than an inherent weakness of the method. Several studies have proposed adaptive cooling methods to reduce parameter sensitivity. Zhan et al. (2016) developed List-Based Simulated Annealing for the Traveling Salesman Problem. Instead of using one fixed cooling schedule, the method adjusted its temperature based on the search behavior. This allowed faster cooling early in the search and slower cooling during later refinement. The list-based approach reduced dependence on a fixed initial temperature and cooling coefficient. Zhan et al. (2016) reported competitive results on Traveling Salesman Problem instances ranging from 48 to 85,900 cities. Their experiments also examined several neighborhood operators. Inversion was the strongest individual operator, insertion generally performed second, and simple swapping performed worst. A hybrid neighborhood that combined inversion, insertion, and swapping produced the strongest overall results. These findings demonstrate that Simulated Annealing performance depends not only on cooling but also on the quality of the neighborhood moves.

Hybrid methods have also incorporated Simulated Annealing into population-based algorithms. Mhamdi et al. (2011) combined Particle Swarm Optimization, Simulated Annealing, and Tabu Search for microwave-image reconstruction. Particle Swarm Optimization provided population-level movement, while Simulated Annealing and Tabu Search locally improved different portions of the swarm. The complete three-algorithm hybrid produced lower reconstruction error than standard PSO and the two partial hybrids. The authors attributed the improvement to the combination of global exploration, probabilistic escape, and

memory-guided local refinement. Simulated Annealing has several important advantages. Its basic structure is conceptually simple, it does not require gradient information, and it can be adapted to continuous, nonlinear, discontinuous, and combinatorial objective functions. Its acceptance of worsening moves gives it a direct mechanism for leaving local optima. The neighborhood operator can also be designed according to the structure of the problem, allowing the method to be applied to route optimization, scheduling, VLSI design, network-flow parameter tuning, feature selection, and inverse reconstruction.

However, Simulated Annealing also has important limitations. Its effectiveness is sensitive to the starting temperature, cooling rate, number of transitions, objective-function scale, and stopping rule. Poorly selected parameters may cause premature convergence or excessively long search. Because the method has no explicit memory of previously explored solutions, it may revisit similar regions or spend substantial computation on low quality candidates. Its stochastic nature also creates variability across runs, making repeated independent trials necessary when assessing its performance.

2.3 Tabu Search

Tabu Search is a memory-based metaheuristic developed by Glover (1989). It can accept moves that do not immediately improve the solution and uses its search history to avoid repeatedly returning to the same choices. A basic Tabu Search procedure begins with an initial solution, generates a set of neighboring candidates, and selects the best admissible neighbor. The selected neighbor does not need to improve the current solution. By accepting the best available nonimproving move, the algorithm can move away from a local optimum and continue exploring the solution space.

Unlike Simulated Annealing, Tabu Search does not rely mainly on probabilistic acceptance. It uses a tabu list to restrict recently performed moves or recently encountered solution attributes. When a move is performed, its reversal or selected characteristics are prohibited for a limited number of iterations. These restrictions prevent the search from immediately undoing its decisions and repeatedly cycling among the same solutions. The amount of time a move remains restricted is known as tabu tenure. If the tabu tenure is too short, the search may repeat itself. If it is too long, useful moves may be blocked. The best setting depends on the problem. Watson et al. (2005) analyzed the behavior of Tabu Search on the Job-Shop Scheduling Problem and found that increasing tabu tenure expanded the effective region explored by the search. Larger tenure values substantially increased solution time because the search was forced farther from previously visited areas. The authors recommended selecting the smallest tenure that still prevents cycling and stagnation. Their findings show that stronger restrictions do not automatically produce better search performance. Tabu restrictions may occasionally prevent an exceptionally strong move.

An aspiration criterion allows a tabu move when it satisfies a sufficiently desirable condition. The most common aspiration rule permits a tabu move when it produces a solution better than the best solution found during the search. This prevents memory restrictions from blocking a new global best candidate. Tabu Search may incorporate several levels of adaptive memory. Short term memory prevents cycling by recording recently used moves or attributes. Intermediate-term memory supports intensification by identifying features that frequently appear in high-quality solutions. The search can then concentrate on regions containing those characteristics. Long-term memory supports diversification by recording how often particular decisions have been selected. Frequently used features may be penalized so that the algorithm is encouraged to examine less-visited areas. Glover (1989) also described more advanced uses of memory that allow Tabu Search to explore areas that ordinary local search may avoid. This shows that Tabu Search is more than simply a list of forbidden moves. Watson et al. (2005) studied how Tabu Search behaves in job shop scheduling. They found that the search often stays close to local optima but can move away from one and continue toward another promising solution. The study also found that problem difficulty was strongly related to the effective width of the region explored rather than the total number of feasible schedules. The initial solution had limited influence on difficult instances unless it began extremely close to an optimum. This suggests that the search dynamics and memory structures may become more important than starting quality as the problem becomes harder. Tabu Search has been widely applied to discrete problems such as scheduling, routing, feature selection, and production planning. Its explicit memory helps prevent short-term repetition, while best admissible movement provides strong local exploitation. Aspiration rules and diversification mechanisms also allow the method to escape restrictive search patterns.

Despite these strengths, Tabu Search has several limitations. Its performance depends heavily on the design of the neighborhood, the tabu attributes, the tabu tenure, the candidate list size, and the aspiration rule. An ineffective neighborhood may fail to produce meaningful improvements even when the memory system is well designed. Evaluating several neighboring candidates at every iteration may also be computationally expensive, especially when each candidate requires a complex simulation or model evaluation. Tabu Search also needs to be adapted to each problem because a useful move in routing may be very different from a useful move in scheduling or feature selection.

2.4 Particle Swarm Optimization

Particle Swarm Optimization was introduced by Kennedy and Eberhart (1995). It is a population-based metaheuristic inspired by collective behavior in bird flocks, fish schools, and other socially interacting groups. Each particle represents one possible solution and learns from both its own best result and the best result found by the swarm. A particle's next movement is influenced by its previous direction, its own best result, and the best result found by the swarm. The personal component encourages particles to use their own successful experience, while the social component encourages them to learn from the strongest solution discovered by the group. Momentum and personal attraction support exploration because particles retain their previous movement and may return to locations they individually found useful. Social attraction supports exploitation because the population moves toward the strongest known solution. If social influence becomes too strong, the swarm may converge prematurely around a local optimum. If personal influence becomes too strong, particles may move too independently and fail to benefit from collective information. Kennedy and Eberhart (1995) observed that a reasonable balance between personal and social influence produced the most effective search behavior.

Later PSO developments introduced an inertia weight that controls how strongly particles preserve their previous velocity. A high inertia weight encourages broader movement, while a lower value promotes refinement around known solutions. Many implementations gradually reduce inertia during the search so that the swarm begins with stronger exploration and ends with stronger exploitation. Other parameters control how strongly particles follow their own experience or the best solutions found by the

swarm. Sengupta et al. (2019) emphasized that no PSO parameter configuration is universally optimal. Performance depends on the objective function, dimensionality, topology, population size, and diversity of the swarm. Parameter values should therefore be selected according to the problem and validated experimentally rather than treated as universal constants. PSO performance is also affected by the swarm topology. PSO can also control how widely information is shared between particles. Sharing information quickly can speed up convergence, but it can also cause the swarm to follow a poor solution too early. Slower information sharing may preserve more diversity and reduce the risk of premature convergence. Standard PSO was developed for continuous optimization. Route planning, scheduling, and feature selection are discrete or combinatorial, so the concepts of position and velocity must be adapted. Discrete PSO methods replace the original continuous movement with problem-specific ways of representing choices, such as binary feature selection or ordered job and route sequences. Alharkan et al. (2020) used Geometric Particle Swarm Optimization for a scheduling problem involving two parallel machines and one shared setup server. Instead of using standard continuous movement, the algorithm generated new job orders using information from the current solution and other strong solutions. Huang et al. (2016) developed a modified discrete PSO for multi-objective flexible job-shop scheduling. Their representation contained an operation-sequence vector and a machine-assignment vector. The algorithm used specialized operators to keep schedules valid while balancing makespan and workload.

PSO has also been adapted to feature selection.

Zhang et al. (2017) represented each feature through a continuous probability value and selected the feature when the value exceeded a threshold. Xie et al. (2021) used enhanced continuous PSO models whose positions were decoded into binary feature subsets. These studies show that discrete applications often preserve the social-learning principle of PSO while replacing its original movement equation with problem-specific operators.

Multi-objective PSO can keep several strong trade-off solutions instead of selecting only one overall best result.Zhang et al. (2017) used an external archive for multi-label feature selection, where the objectives were to minimize Hamming loss and minimize the number of selected features. Adaptive mutation and local differential learning were used to improve exploration and Pareto-front coverage. PSO is frequently hybridized with other optimization methods to address premature convergence and weak local refinement. Hybrid approaches combine PSO with other optimization or local-search methods, including Simulated Annealing and Tabu Search. Sengupta et al. (2019) identified hybridization as one of the most active areas of PSO research. However, they also noted that hybrid methods often contain many interacting mechanisms and parameters, making it difficult to determine which component produced the observed improvement.

Particle Swarm Optimization offers several advantages. It performs population-based exploration, shares information rapidly, can be implemented using relatively simple operations, and is compatible with parallel processing. Its population allows several areas of the search space to be examined simultaneously. However, it is vulnerable to premature convergence, stagnation, and loss of diversity. Its behavior depends on the quality of the leaders and the selected movement parameters. Discrete problems also require specialized representations and operators that may substantially change the behavior of the original algorithm.

2.5 Benchmark Optimization Problems

The present study focuses on three major categories of optimization problems: route optimization, job scheduling, and machine-learning feature selection. These problems were selected because they require different kinds of solutions. Routing focuses on the order of locations, scheduling involves assigning and ordering jobs, and feature selection involves choosing which features to keep. Using several problem categories may provide a broader understanding of the algorithms than testing them on one application alone. An algorithm that performs effectively in a continuous or permutation-based space may behave differently in a binary subset space. Differences in solution representation, neighborhood structure, objective cost, and constraint handling may favor different search strategies.

2.6 Route Optimization

Route optimization aims to find an efficient path between locations. A common example is the Traveling Salesman Problem, which asks for the shortest route that visits every city once and returns to the starting point. The quality of a route is usually measured by its total distance. The number of possible routes increases factorially with the number of cities. Consequently, exhaustive enumeration becomes impractical for large instances. This has made the Traveling Salesman Problem a common benchmark for evaluating exact algorithms, heuristics, and metaheuristics. Grabusts et al. (2019) applied Simulated Annealing to an eight-location route involving dairy enterprises in Belarus. A 2-opt neighborhood generated new solutions by reversing a selected segment of the route. The reported route stabilized at approximately 648.69 kilometers. The study demonstrated a practical implementation of SA for route construction, but the small instance size limited the strength of its conclusions. The result was not compared with an exact optimum, only one principal stochastic outcome was reported, and the temperature and transition parameters were not documented in sufficient detail. Zhan et al. (2016) provided stronger evidence through List-Based Simulated Annealing. Their method was tested on Traveling Salesman Problem instances ranging from dozens to tens of thousands of cities. The list-based temperature mechanism reduced dependence on manually selected cooling schedules, while a hybrid neighborhood combined inversion, insertion, and swapping. Their results showed that Simulated Annealing could remain competitive on very large routing instances when the cooling method and neighborhood design were carefully constructed. Tabu Search has also been widely applied to route optimization because route changes can be represented naturally through edge exchanges, segment reversals, city insertion, and route swaps. A tabu list may store recently added or removed edges, preventing the algorithm from immediately reversing a route modification. Pirim et al. (2008) reviewed several routing studies and found that Tabu Search frequently produced stronger solution quality than Simulated Annealing and threshold-accepting methods, particularly as problem size increased. However, Tabu Search sometimes required more computation because it evaluated several route candidates at each iteration. Ru (2024) applied Tabu Search to multimodal vehicle logistics routing involving road, rail, and water transport. The model considered transfer costs, transportation time, facility capacity, and route profitability. The company case reported increased profit on most optimized routes. However, the paper contained inconsistencies in its runtime reporting and used classification measures such as recall and ROC-curve area in a routing context without sufficiently explaining their relevance. Particle Swarm Optimization requires substantial adaptation for route problems because a route is a permutation rather than a continuous vector. Because routes are ordered sequences, PSO

must be modified so that particles can represent and change valid routes. Sengupta et al. (2019) reviewed several PSO hybrids for route optimization, many of which combined PSO with local route-improvement methods or other optimization algorithms such as 2-opt or Simulated Annealing. These combinations allow PSO to examine several route regions simultaneously while relying on local operators to refine individual routes. Overall, SA provides flexible route changes, TS uses memory to avoid repeating recent moves, and PSO can explore several routes at once but requires more modification. Hybrid local search often strengthens all three methods. A fair comparison should therefore measure route quality, objective evaluations, runtime, convergence speed, and consistency across repeated runs.

2.7 Job Scheduling and Machine Allocation

Job scheduling assigns operations to machines and determines their processing order. Common goals include finishing all jobs sooner, reducing delays, and balancing work between machines. Scheduling becomes increasingly difficult when it includes multiple machines, alternative machine assignments, setup times, shared servers, precedence constraints, due dates, or limited capacity. In the classical Job-Shop Scheduling Problem, each job contains several operations that must be processed on specified machines in a predetermined order. The objective is commonly to minimize makespan, which is the completion time of the final operation. Watson et al. (2005) showed that the search space contains many local and near-local-optimal schedules. Watson et al. (2005) found that more structured scheduling problems could be harder because Tabu Search had to explore a wider range of possible schedules. In Flexible Job-Shop Scheduling, some jobs can be handled by more than one machine. The scheduler must therefore decide both which machine should handle each job and in what order. Huang et al. (2016) treated this as a multi-objective problem and minimized makespan, total machine workload, and maximum individual machine workload. Their modified discrete PSO produced several strong scheduling trade-offs on standard benchmark problems. Alharkan et al. (2020) studied a scheduling problem involving two identical parallel machines served by one setup server. Each job required a setup period and a processing period. Since only one server could perform setup at a time, the algorithm had to coordinate server availability with machine availability. The study compared Tabu Search, Geometric Particle Swarm Optimization, Simulated Annealing, a Genetic Algorithm, and Iterated Local Search. Tabu Search produced the lowest worst-case average error ratio and most frequently reached the theoretical lower bound for large instances. GPSO generally ranked second, while Simulated Annealing and the Genetic Algorithm performed strongly on small instances but deteriorated as problem size increased.

Jwo et al. (2023) applied Tabu Search to manufacturing-order sequencing in an aircraft-industry work center. Their model considered daily capacity, due dates, processing times, remaining lead time, and unfinished orders carried into subsequent days. The neighborhood swapped delayed manufacturing orders with other orders. When initialized using First In, First Out, the method reached the theoretical lower bound for the number of delayed orders on all five tested days. Its total expected delay also remained close to the ideal reference values. Simulated Annealing represents a schedule as one current sequence or assignment. Neighboring schedules may be generated by swapping jobs, inserting an operation, reversing part of a sequence, or changing a machine assignment. Its probabilistic acceptance rule allows the search to leave locally optimal schedules. However, difficult scheduling instances may require many evaluations before the cooling schedule produces sufficiently focused refinement. Alharkan et al. (2020) found that Simulated Annealing reached the lower bound in all eight-job cases but did not maintain the same performance as the number of jobs increased. Tabu Search is especially common in scheduling because job exchanges, operation movements, and machine reassignments naturally define neighborhoods. Its memory can track recent job swaps or machine assignments so that the search does not immediately repeat the same decisions. Tabu Search generally performs well when its job-swap rules are useful and its tabu settings prevent repetition without blocking too many good moves. Particle Swarm Optimization must be modified for scheduling because continuous particle movement does not naturally preserve valid job permutations or operation precedence. Discrete PSO scheduling methods replace velocity movement with crossover, swaps, probability vectors, and decoding procedures. Their population based nature allows several schedule regions to be examined simultaneously, but maintaining validity requires more complex operators than standard PSO. The scheduling literature generally indicates that Tabu Search performs strongly on medium and large combinatorial instances. Discrete PSO variants can also scale effectively when their representation preserves feasible schedules and maintains diversity. Simulated Annealing may perform well on small or moderately sized instances but can require longer computation on difficult problems. These conclusions remain dependent on the scheduling formulation, neighborhood operators, evaluation limits, and parameter settings used by each study.

2.8 Machine-Learning Feature Selection

Feature selection is the process of identifying a subset of relevant variables from a larger feature set. For a dataset containing n features, there are 2^n possible subsets. Even a moderate number of variables therefore creates a very large combinatorial search space.

Feature selection aims to remove unnecessary or redundant features while keeping good predictive performance. This can also reduce training time and make the model easier to understand. Removing unnecessary features may also reduce overfitting, particularly when the number of available training samples is small relative to the number of variables. Feature-selection methods can use simple statistical measures or repeatedly test different feature subsets using a machine-learning model. The second approach can capture interactions between features but usually requires more computation. Allvi et al. (2020) applied Simulated Annealing to feature selection for learning-to-rank systems. A state represented a feature subset of fixed size, while a neighboring state was created by replacing selected feature indices. The study used LambdaMART and measured the quality of each feature subset using a ranking-performance score. The search was repeated for each possible subset size. The authors reported that subsets containing approximately 18 to 39 features could match or exceed the performance of the complete feature sets across six benchmark datasets. Their findings suggested that some original features were redundant or harmful. However, the study used random feature selection as its main baseline, did not report sufficiently detailed repeated-run statistics, omitted several important Simulated Annealing parameters, and appeared to use test performance during feature selection. This may have made the reported results appear better than they would be on completely unseen data. Zhang and Sun (2002) applied Tabu Search to both fixed-size and minimum-size feature selection. The method tested feature subsets by adding, removing, or replacing selected features. The method was compared with several common feature selection methods, including Genetic Algorithms and Branch and

Bound. In synthetic experiments involving 30, 60, and 100 features, Tabu Search generally produced better best, mean, and worst objective values than the Genetic Algorithm under similar or lower evaluation costs. In a 20-feature fixed-size task, Tabu Search reached the Branch-and-Bound optimum in all 20 runs when selecting 10 features. The study also found that Tabu Search performance depended on its tabu settings and the size and design of the candidate neighborhood. Xie et al. (2021) developed two enhanced PSO methods, PSOVA1 and PSOVA2, for wrapper based feature selection. Their evaluation used a K-Nearest Neighbor classifier and combined classification accuracy with feature reduction. Accuracy received substantially greater weight than subset size. PSOVA1 and PSOVA2 added several mechanisms intended to improve exploration, strengthen promising particles, and reduce premature convergence. Across 13 datasets containing from 30 to 22,283 features, the proposed methods achieved the highest reported accuracy and F-score among the compared algorithms. PSOVA2 significantly outperformed the baseline methods on eight datasets. However, the proposed algorithms were highly complex. Their improvements resulted from PSO combined with several additional mechanisms, making it difficult to determine how much of the benefit came from the basic swarm structure itself. Zhang et al. (2017) treated multi-label feature selection as a two-objective problem. The algorithm simultaneously minimized Hamming loss and the number of selected features. It kept several strong trade off solutions and used additional mechanisms to preserve diversity during the search. Across six datasets, the method generally produced better trade-off solutions than the comparison methods. The study demonstrated that feature selection does not always need to be converted into one weighted objective. Multi-objective optimization can preserve several useful trade-offs between predictive performance and dimensionality. The feature-selection literature indicates that all three algorithms can search large subset spaces. Simulated Annealing provides a simple mechanism for moving among subsets but requires careful cooling and repeated model evaluations. Tabu Search provides explicit memory and strong neighborhood control. Particle Swarm Optimization can examine several feature subsets at once, but it may lose diversity and requires a suitable binary or probability-based representation. A fair comparison of feature-selection algorithms should evaluate predictive performance, subset size, runtime, objective evaluations, convergence, and stability across repeated trials. The same classifier, preprocessing method, training-validation-test partition, and performance metric should be used for all algorithms. Candidate subsets should be selected using training or validation data, while test data should be reserved for final evaluation.

2.9 Comparative Studies of Simulated Annealing, Tabu

Search, and Particle Swarm Optimization Several reviewed studies directly compared two or more of the selected metaheuristics. These comparisons provide evidence regarding solution quality, convergence, computational efficiency, and search behavior, although their findings remain dependent on the problem and experimental conditions.

Zhang and Nicholson (2018) compared Boltzmann Simulated Annealing, Very Fast Annealing, Tabu Search, and Particle Swarm Optimization for tuning a parameter in the Parameterized Dynamic Slope Scaling Procedure for Fixed-Charge Network Flow. The study included 180 generated network instances. All four methods improved most of the tested instances, and none was statistically superior in final solution quality. The main difference was computational efficiency. Particle Swarm Optimization produced the greatest improvement per iteration and was fastest on small and large instances. Tabu Search remained competitive on the largest instances, while both Simulated Annealing variants required substantially more computation. The study demonstrates that similar final objective values may conceal meaningful differences in convergence and evaluation efficiency. Youssef et al. (2001) compared a Genetic Algorithm, Simulated Annealing, and Tabu Search on VLSI floorplanning under a shared budget of 5,000 objective evaluations. The objective combined circuit area, wire length, and delay through a fuzzy membership score. Across five circuits, the reported ranking was Tabu Search first, Genetic Algorithm second, and Simulated Annealing third. Tabu Search reached strong solutions quickly, while the Genetic Algorithm later plateaued and Simulated Annealing required more evaluations in weaker regions. However, the cost-inflation experiment showed that its poor performance was partly caused by a mismatch between the fuzzy objective scale and the temperature schedule. This emphasizes that algorithm comparisons must consider whether each method has been calibrated appropriately. Alharkan et al. (2020) compared Tabu Search, Geometric Particle Swarm Optimization, Simulated Annealing, a Genetic Algorithm, and Iterated Local Search on the two-machine single-server scheduling problem. Tabu Search produced the lowest worst-case average error and most frequently reached the theoretical lower bound on large instances. Geometric Particle Swarm Optimization ranked second overall. Simulated Annealing and the Genetic Algorithm were competitive for smaller instances but deteriorated as the number of jobs increased. Pirim et al. (2008) reviewed many studies comparing Tabu Search with Simulated Annealing, Genetic Algorithms, Particle Swarm Optimization, and Ant Colony Optimization. Their findings indicated that Tabu Search frequently performed strongly in scheduling, routing, facility location, and production planning. However, they also identified cases in which Simulated Annealing produced better schedules or Genetic Algorithms performed better because their representation reduced the number of candidate solutions. Algorithm rankings also changed according to whether comparisons were controlled by runtime, evaluated solutions, or unrestricted convergence. The review therefore supports a problem-dependent rather than universal interpretation of performance.

Mhamdi et al. (2011) compared standard PSO with several hybrid forms in microwave imaging. In one experiment, the full PSO-SA-TS hybrid produced the lowest reconstruction error compared with standard PSO and the partial hybrids. The complete hybrid also reduced computation time in another experiment. The authors attributed the improvement to the combination of Particle Swarm Optimization's population-level exploration, Simulated Annealing's probabilistic local escape, and Tabu Search's memory-based refinement. However, the study did not compare full standalone implementations of SA and TS under the same reconstruction problem, so its findings primarily support the value of hybridization rather than a direct ranking of the three independent methods. The comparative literature shows that no one algorithm consistently dominates all problem categories. Tabu Search frequently performs strongly on neighborhood-intensive combinatorial problems. Particle Swarm Optimization often demonstrates strong efficiency and scalability when an appropriate representation is available. Simulated Annealing can produce competitive final quality but may require more evaluations and careful calibration of temperature and objective scale. Hybrid methods may combine complementary strengths, although they also increase implementation complexity and parameter

interactions.

2.10 Evaluation Criteria Used in Previous Studies

The reviewed studies used different evaluation measures depending on the optimization problem. Studies commonly measured solution quality using best and average results, error from a known solution, or how often an algorithm reached the optimum. Reporting only the best result is insufficient because it does not show how consistently a stochastic algorithm performs. Computational efficiency was measured through CPU time, number of iterations, number of objective-function evaluations, improvement per iteration, and convergence speed. Raw iteration counts should be interpreted cautiously because the meaning of one iteration differs among the algorithms. One Tabu Search iteration may evaluate several candidates, one Simulated Annealing transition may evaluate only one neighbor, and one PSO iteration evaluates the complete swarm. Equal numbers of objective evaluations or equal runtime budgets may therefore provide a fairer comparison than equal iteration counts. Stability should be evaluated through repeated independent runs. Repeated runs can be summarized using average results, variation, and success frequency. Repeated trials are particularly important for Simulated Annealing and Particle Swarm Optimization because their search paths depend heavily on random choices. Tabu Search may also require repeated trials when it uses random initialization, neighborhood sampling, or tie-breaking.

Multi-objective studies use measures that consider both how good the trade-off solutions are and how well they are distributed. These metrics assess not only convergence toward the Pareto front but also the diversity and distribution of trade-off solutions. Feature-selection studies use classification accuracy, F-score, precision, recall, Hamming loss, and Normalized Discounted Cumulative Gain. The selected metric should match the predictive task and should be evaluated on data not used to guide the optimization process. The number or percentage of selected features should also be reported so that improvements in predictive performance are not achieved solely by retaining nearly every variable. Several studies also used statistical tests to determine whether performance differences were likely to be meaningful rather than caused by random variation. The statistical method used should still match the type of results being analyzed.

2.11 Issues and Limitations in Existing Literature

Parameter sensitivity is a recurring issue across all three algorithms. All three algorithms depend on several parameter choices. SA is sensitive to its temperature and cooling settings, TS depends on its memory and neighborhood settings, and PSO depends on its swarm and movement parameters. Parameter values that perform well for one problem may perform poorly for another. Many studies select parameters heuristically or copy values from earlier literature without systematic tuning. This makes it difficult to determine whether poor performance results from the algorithm or from unsuitable parameter selection. Too much tuning on the same test problems may also make results look better than they would on new problems. Experimental conditions are often inconsistent across studies. Algorithms may be implemented using different datasets, hardware, programming languages, computational budgets, stopping rules, and levels of optimization. Some studies compare a newly implemented algorithm with numerical results copied from earlier papers instead of rerunning all methods in the same environment. These differences make it difficult to know whether the results are caused by the algorithms themselves or by the way the experiments were set up. Repeated-run reporting is also frequently inadequate. Some studies report only one run, one best result, or a mean without standard deviation. This is insufficient for stochastic optimization because one unusually favorable run may create a misleading impression of performance. Reliable evaluation requires several independent trials and measures of variation.

Several reviewed studies were conducted on small or synthetic problems. Grabusts et al. (2019) used only eight route locations, which is small enough for exact enumeration. Zhang and Sun (2002) used synthetic variable-size feature-selection problems containing 30 to 100 features, which are modest relative to modern datasets containing thousands of variables. Other studies used randomly generated scheduling instances or synthetic error models. Such experiments provide controlled conditions but may not represent the uncertainty, dynamic conditions, and operational constraints of real applications. Weak or inappropriate baselines are another concern. Some studies compared a proposed method only with its immediate predecessor, random selection, or previously published results. Niño et al. (2012) constructed the reference Pareto front from the outputs of the same two algorithms being compared, weakening the interpretation of its perfect recovery results. Allvi et al. (2020) primarily compared Simulated Annealing with random feature selection rather than stronger established feature-selection methods; its unclear use of test data during optimization further limits the independence of the reported evaluation. Some papers omitted critical implementation details such as random seeds, exact initial and final temperatures, complete cooling schedules, neighborhood sampling procedures, archive limits, and precise stopping conditions. Missing details reduce reproducibility and make it difficult to determine whether performance differences reflect algorithmic behavior or implementation choices. Hybrid methods present an additional interpretive challenge. Hybrid algorithms often combine several additional search mechanisms, making it difficult to know which part actually caused the improvement. Increased complexity also creates more parameters and a greater risk of overfitting the algorithm to the selected benchmark set.

2.12 Synthesis of the Reviewed Literature

The reviewed studies establish that Simulated Annealing, Tabu Search, and Particle Swarm Optimization are all capable of solving difficult optimization problems, but they rely on substantially different search mechanisms. Simulated Annealing maintains one current solution and uses temperature-dependent acceptance to control its movement. Its major strength is its ability to cross inferior regions and escape local optima. Its main weakness is its sensitivity to cooling settings and the way the objective function is scaled. A carefully designed neighborhood and temperature schedule can make the method competitive on large problems, while poor calibration can cause excessive computation in low-quality regions. Tabu Search also works with one current solution, but it uses memory to guide which nearby solution should be tried next. It often performs well in scheduling, routing, and feature-selection problems because its memory can guide the search around nearby solutions. Its effectiveness depends on meaningful neighborhood operators, appropriate tabu tenure, and sufficiently informative candidate evaluation. Excessive restrictions or poorly selected tabu attributes may weaken performance. Particle Swarm Optimization maintains a population of candidate solutions and uses personal and social learning. Its population can examine several regions at the same time, and its simple information-sharing mechanism can produce rapid

convergence. However, the swarm may lose diversity and converge prematurely around a local optimum. Discrete problems also require specialized representations that may substantially alter the original continuous PSO mechanism. Across direct comparisons, no algorithm consistently dominates every problem. Tabu Search often performs well on problems where useful nearby solutions can be created through simple changes. Particle Swarm Optimization often provides efficient convergence and scalability when its discrete representation is appropriate. Simulated Annealing can produce competitive solutions but may require more evaluations and greater care in parameter calibration. Hybrid algorithms can combine the strengths of several methods, but their added complexity makes it harder to know which part actually caused the improvement. The relative performance of the algorithms depends on problem type, problem size, encoding, neighborhood or movement design, parameter values, evaluation budget, stopping conditions, and implementation quality. One algorithm should therefore not be declared universally superior based on isolated experiments. A controlled comparison should examine both final solution quality and the process by which that quality is obtained.

2.13 Research Gap

Previous studies have applied Simulated Annealing, Tabu Search, and Particle Swarm Optimization to route optimization, scheduling, and feature selection. However, most studies focus on one application domain and use highly problem-specific implementations. Existing findings remain difficult to compare because studies use different problems, datasets, algorithm versions, parameters, and evaluation methods. Some comparisons measure runtime, while others use iteration count or objective evaluations. Several studies do not report repeated-run statistics or formal significance tests. Others compare heavily modified or hybrid algorithms with basic implementations, making it difficult to determine whether observed improvements result from the fundamental algorithm or from additional operators. The three selected problem categories also have substantially different structures. Route optimization commonly uses permutations of locations. Scheduling combines sequencing, resource assignment, precedence, and capacity constraints. Feature selection uses binary subsets and may require an expensive predictive-model evaluation for every candidate. These differences may favor different search mechanisms. Tabu Search may benefit from clearly defined discrete neighborhoods, Particle Swarm Optimization may benefit from population diversity, and Simulated Annealing may benefit from flexible stochastic movement. Existing findings remain difficult to compare because studies use different problems, datasets, algorithm versions, parameters, and evaluation methods. This study addresses the gap by bringing together research on Simulated Annealing, Tabu Search, and Particle Swarm Optimization across route optimization, job scheduling, and machine-learning feature selection. The main method will be a structured comparison of the reviewed studies, supported when feasible by small sample tests. The goal is not to identify one universal winner, but to understand where each algorithm performs well and what practical strengths and weaknesses should be considered when choosing between them.

2.14 Significance of the Present Study

The study has theoretical, methodological, and practical significance. First, the study compares three algorithms that search for solutions in different ways. Looking at them across routing, scheduling, and feature selection may help explain why certain algorithms are better suited to some problems than others. The study also highlights how differences in datasets, parameters, stopping rules, and evaluation methods can affect comparisons between algorithms. This approach addresses several weaknesses identified in previous research, including inconsistent budgets, incomplete parameter descriptions, missing variation measures, and comparisons conducted under different experimental environments. From a practical perspective, the findings may support algorithm selection across several domains. Route planning practitioners may prioritize distance reduction, runtime, and route stability. Production and manufacturing planners may prioritize makespan, machine utilization, tardiness, and schedule feasibility. Machine-learning practitioners may prioritize predictive performance, dimensionality reduction, and training cost. By comparing the algorithms across different problems, the study may help researchers and practitioners choose a method based on what the problem actually requires instead of assuming that one algorithm is always best.

3. PROPOSED COMPARATIVE APPROACH

The primary method will be structured comparison of the reviewed studies. Evidence will be grouped by application area and interpreted according to the measures actually reported by each source. Because the studies use different datasets, implementations, budgets, and stopping rules, their numerical results will not be compared as if they came from the same experiment.

The sample tests, when completed, will use simplified versions of route optimization, job scheduling, and feature selection. The tests should document the selected representation, algorithm parameters, stopping conditions, and evaluation measure. Repeated runs and basic statistics may be used where practical, but the sample tests will remain small demonstrations rather than full-scale experiments.

The final framework will summarize where studies generally agree, where their findings differ, and where the available evidence is too limited to make a clear conclusion. Special attention will be given to weak baselines, incomplete parameter reporting, inappropriate use of test data, small problem instances, and comparisons between heavily modified algorithms and basic implementations.

4. FORMAL AND THEORETICAL ANALYSIS OF SIMULATED ANNEALING, TABU SEARCH, AND PARTICLE SWARM OPTIMIZATION

Sections 2 and 3 characterized Simulated Annealing (SA), Tabu Search (TS), and Particle Swarm Optimization (PSO) primarily through reported empirical behavior. This section complements that treatment with a formal characterization of each algorithm as a search procedure over a defined state space, including its move/acceptance mechanism, per-iteration computational cost, and theoretical convergence properties.

4.1 Computational Characterization

Simulated Annealing. SA is a single-solution, stochastic local-search procedure. Let S denote the solution space and f: S → R (the real numbers) the objective function. At

iteration k, a candidate $s_k \in S$ generates a neighbor s' via a problem-specific move operator $N(s_k)$. The neighbor is accepted according to

$P(accept) = 1$, if $f(s') \le f(s_k)$; $P(accept) = \exp(-(f(s') - f(s_k)) / T_k)$, otherwise,

where T_k is the temperature at iteration k, typically updated by geometric cooling $T_{k+1} = \alpha \cdot T_k$, with $0 < \alpha < 1$. SA is therefore a Markov chain over S whose transition probabilities are governed entirely by T_k.

Tabu Search. TS is also single-solution but replaces probabilistic acceptance with deterministic best-admissible selection guided by short-term memory. At iteration k, TS generates a full candidate set $N(s_k) = \{s'_1,...,s'_m\}$ and selects

$s_{k+1} = \text{argmin}_{s' \in N(sk) \setminus Tabu} f(s')$, unless an aspiration criterion (typically $f(s') < f(s_{best})$) overrides the tabu restriction. The tabu list is a finite-memory structure of tenure t that forbids recently used moves or attributes for t iterations.

Particle Swarm Optimization. PSO is a population-based procedure over n particles, each with position $x_i(t)$ and velocity $v_i(t)$. The update rule is

$v_i(t+1) = w \cdot v_i(t) + c_1 r_1 (pbest_i - x_i(t)) + c_2 r_2 (gbest - x_i(t))$;

$x_i(t+1) = x_i(t) + v_i(t+1)$,

where w is the inertia weight, c_1, c_2 are the cognitive and social coefficients, and r_1, $r_2 \sim U(0,1)$. PSO is a population-level stochastic dynamical system rather than a single-chain Markov process, which is the formal source of its parallel-exploration property discussed in §2.4.

4.2 Per-Iteration Computational Complexity

Table 1 reports the dominant per-iteration cost of each algorithm, where d is the solution dimensionality (e.g., number of cities, operations, or candidate features), m is the TS candidate-list size, and n is the PSO swarm size. These costs explain why raw iteration counts are not comparable across algorithms (§2.10): a single TS iteration performs O(m) objective evaluations, a single PSO iteration performs O(n), while a single SA transition performs exactly one.

Algorithm	Evals / iter.	Update cost / iter.	Memory
SA	O(1)	O(d) (one move)	O(1)
TS	O(m)	O(m·d)	O(t)
PSO	O(n)	O(n·d)	O(n·d)

Table 1. Dominant per-iteration cost by algorithm.

Under the evaluation-budget-fair protocol used in this study (§5), these differences mean that TS with m = 20 performed roughly 20× fewer “iterations” than SA for the same number of objective evaluations — consistent with the early plateaus TS exhibits in Figures 1–3 once its fixed neighborhood has been exhausted relative to the remaining budget.

4.3 Convergence Properties and Theoretical Guarantees

SA. Under an idealized logarithmic cooling schedule $T_k = c / \log(k)$ and a fully connected neighborhood structure, SA is proven to converge in probability to a global optimum (the theoretical basis underlying Kirkpatrick et al., 1983 [8]). This guarantee does not extend to the finite-time geometric schedules used in practice, including in this study; a finite geometric schedule provides no formal convergence guarantee, only an empirical tendency to improve, which is consistent with why this study's SA results were highly sensitive to how the schedule was scaled to the evaluation budget (§5.3).

TS. Tabu Search carries no probabilistic convergence guarantee. Because it uses deterministic best-admissible selection over a finite, fixed-size neighborhood, its long-run behavior is bounded by neighborhood coverage rather than asymptotic convergence: once every reachable candidate in N(s) has been evaluated relative to the current tabu state, further iterations cannot discover new information unless the neighborhood itself is redefined or diversified (Glover, 1989 [3]). This formally explains the early plateau behavior observed for TS in this study's TSP and Feature Selection results.

PSO. Standard PSO likewise has no formal guarantee of convergence to a global optimum. Particle trajectories converge to a stationary point that is a weighted average of personal-best and global-best positions; this point coincides with the true optimum only when swarm diversity is preserved long enough to have sampled its neighborhood. Premature loss of diversity — the mechanism Sengupta et al. (2019 [13]) identify as PSO's central weakness — is therefore a convergence-theoretic property, not merely an empirical tendency.

4.4 Representation and State-Space Considerations

The three benchmark domains induce structurally different state spaces. TSP defines a permutation space of size $(d-1)!/2$ for d cities; JSSP defines a joint sequencing/assignment space; and Feature Selection defines a binary hypercube $\{0,1\}^d$ of size 2^d. SA and TS operate natively on any of these spaces because their move operators are problem-defined rather than fixed. Standard PSO, by contrast, is defined for continuous R^d, so applying it to permutation or binary spaces requires an indirect encoding (e.g., random-key decoding for permutations, or sigmoid thresholding for binary vectors). This encoding step is a formal, not merely practical, source of distortion: the continuous velocity update in §4.1 optimizes smoothly over R^d, but the decoding function that maps R^d back onto S is generally discontinuous and non-injective, which is the underlying reason PSO's empirical performance gap was largest on the two permutation-encoded problems (TSP, JSSP) and smallest

on the natively binary-encoded Feature Selection problem (§5.4).

4.5 Summary of Theoretical Properties

Property	SA	TS	PSO
State representation	Single solution	Single solution	Population (n solutions)
Acceptance rule	Stochastic (temperature)	Deterministic best-admissible	Deterministic velocity update
Formal convergence guarantee	Yes, under log. cooling (impractical)	None	None
Native domain	Any (operator-defined)	Any (operator defined)	Continuous R^d
Requires indirect encoding for combinatorial problems	No	No	Yes

Table 2. Formal properties of SA, TS, and PSO.

5. RESULTS, DISCUSSION, COMPARATIVE EVALUATION FRAMEWORK, AND CONCLUSION

This section reports the results of the small-scale sample tests referenced in §3, evaluates the degree to which those results support or complicate the expectations set out in §2, consolidates the findings into a Comparative Evaluation Framework, and answers the study's research questions. All results reflect 10 independent runs per algorithm per problem under a shared, evaluation-budget-fair protocol (§2.10).

5.1 Summary of Results

Problem	SA (mean)	TS (mean)	PSO (mean)	Winner
TSP (50 cities, 20,000 evals)	**581.71**	694.37	1268.57	SA
JSSP (FT06, 20,000 evals, optimum=55)	**55.00**	**55.00**	57.80	SA = TS
Feature Selection (Breast Cancer, 2,000 evals)	0.0328	**0.0276**	0.0338	TS

Table 3. Mean best objective value per algorithm, 10 runs each.

5.2 Literature Expectations vs. Experimental Results

Table 4 compares the directional expectations drawn from §2 against the observed results.

Expectation (§2)	Result	Match
TS strongest on combinatorial problems (TSP, JSSP)	Tied SA on JSSP; beaten by SA on TSP after SA schedule fix	Partial
SA flexible but parameter-sensitive (Youssef et al., 2001)	SA cooling bug reproduced this failure directly; fixing it flipped SA worst→best on TSP	Strong
PSO struggles with non-native (permutation) encodings	Worst on TSP/JSSP; far more competitive on native binary Feature Selection	Strong

Table 4. Literature expectations against sample-test results.

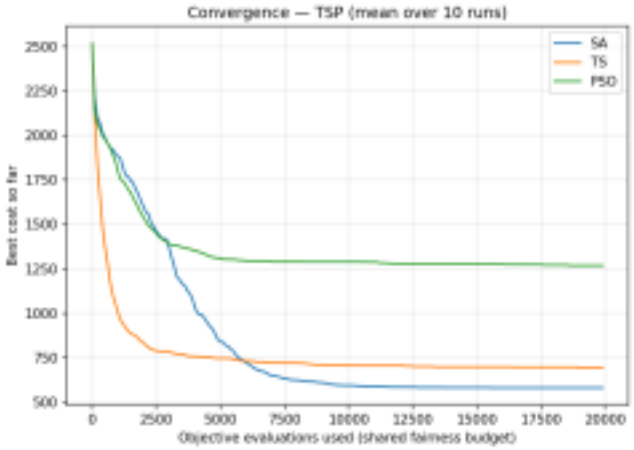

Figure 1. TSP convergence, mean over 10 runs. SA overtakes TS near evaluation 6,000 and continues improving through the full budget, while TS plateaus early.

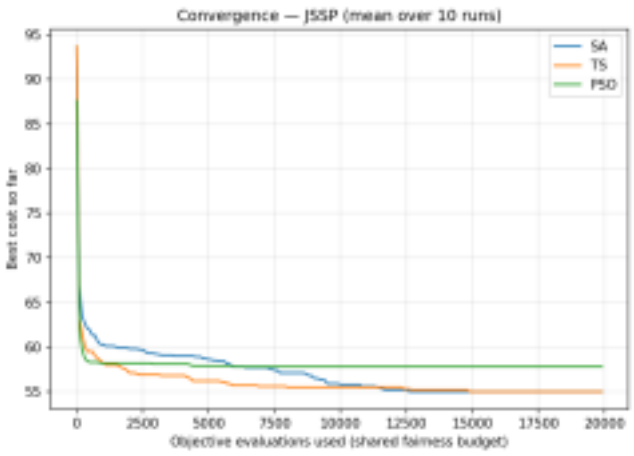

Figure 2. JSSP (FT06) convergence, mean over 10 runs. SA and TS both reach the known optimum (55); PSO plateaus above it.

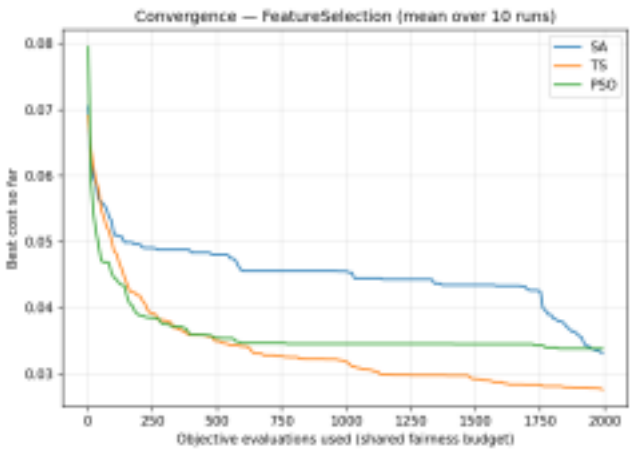

Figure 3. Feature Selection convergence, mean over 10 runs. The spread between all three algorithms is markedly narrower than in Figures 1 and 2.

5.3 Per-Problem Discussion

Route Optimization (TSP, 50 cities). SA finished with a mean tour length of 581.7 against TS's 694.4, roughly a 16% margin in SA's favor — the reverse of the naive "TS wins combinatorial problems" reading of §2.6. The reversal traces to a specific mechanism: TS's candidate-list size (20) was fixed, so each iteration sampled a small, static neighborhood that plateaued early (Fig. 1, ≈evaluation 6,000). SA's cooling schedule, once corrected to span the full budget, kept accepting small uphill moves well past that point, escaping basins TS's tabu memory had already closed off. This bounds rather than overturns the literature's TS-favoring claim: TS's advantage on TSP-style problems appears conditional on a smaller budget or a more generous candidate-list size.

Job Scheduling (JSSP, FT06). SA and TS both reached the known-optimal makespan of 55 on all 10 runs (σ = 0); PSO averaged 57.8 and never reached the optimum. This matches §2.7's TS-favoring expectation closely, with SA matching rather than trailing TS — consistent with FT06's small size (36 operations), where both a well-scheduled SA and a purpose-tuned TS have sufficient budget to fully explore the space. PSO's coarser random-key decoding plausibly explains its weaker, higher-variance result.

Feature Selection (Breast Cancer, 30 features). TS finished first (0.0276), SA close behind (0.0328), PSO third (0.0338) — but the spread across all three (≈0.006) is far narrower, proportionally, than the TSP spread. This is the strongest evidence for §2.4 and §4.4's representation-sensitivity claim: PSO's relative gap to the winner shrank from roughly 118% (TSP) to about 20% (Feature Selection) once the encoding matched its native continuous/binary form. TS's continued edge is plausibly explained by its exhaustive one-bit-flip candidate list, a fine-grained neighborhood well matched to a 30-dimensional binary space.

5.4 Comparative Evaluation Framework

Table 5 consolidates the theoretical characterization (§4) with the empirical behavior observed above, organized by the evaluation criteria §2.10 identified as necessary for a fair comparison.

Criterion	SA	TS	PSO
Search mechanism	Single-solution, stochastic acceptance	Single-solution, memory-guided best move	Population, personal/social learning
Solution quality — permutation problems	Strong if budget-matched (TSP winner)	Strong but plateaus (TSP runner-up; JSSP co-winner)	Weakest — indirect encoding distorts landscape
Solution quality — native-vector problems	Competitive	Best in this test	Most relatively competitive of the three domains
Convergence	Slow early, strong late if not premature	Fast early, early plateau	Fast early, platea u/stagnation
Parameter sensitivity	High (reproduced directly in this study)	Moderate (list size, tenure)	Moderate–high (encoding dominates)
Representation dependency	Low	Low–moderate	High
Best-suited conditions (this study)	Larger budgets, permutation problems, correctly scaled schedule	Small–medium instances, well-sized static neighborhood	Native continuous/binary representation

Table 5. Comparative Evaluation Framework for SA, TS, and PSO.

5.5 Answering the Research Questions

RQ1 (Route optimization). Under a budget-fair protocol, SA produced the best mean tour quality once its cooling schedule was corrected, followed by TS; PSO scaled worst due to its indirect permutation encoding, consistent with the formal encoding argument in §4.4.

RQ2 (Job scheduling). SA and TS were statistically indistinguishable on FT06; PSO was consistently weaker and more variable — the one domain where the literature's TS-favoring expectation held without qualification.

RQ3 (Feature selection). TS produced the best mean result, with SA and PSO close behind; PSO's relative standing improved more than any other algorithm's when moving to its native representation.

RQ4 (Implementation requirements and sensitivity). The sample tests reproduced, first-hand, the exact SA failure mode §2.2 warned about: an under-scaled cooling schedule silently converted SA from best to worst on TSP without any change to the algorithm itself — direct, study-generated evidence that reported weaknesses can be calibration artifacts rather than inherent properties.

RQ5 (Do results support the literature?). Directionally

yes, with two qualifications: TS's combinatorial advantage was budget- and tuning-dependent rather than universal, and PSO's representation sensitivity was the most cleanly and consistently confirmed finding, holding across all three domains in the predicted direction.

5.6 Practical Recommendations

For native binary or continuous-vector problems (e.g., feature selection), TS is a strong, inexpensive default at this scale, with PSO becoming more attractive as dimensionality grows. For small-to-medium classic combinatorial benchmarks (e.g., FT06-scale scheduling), either SA or TS is a safe choice provided SA's cooling schedule is scaled to the evaluation budget. For larger permutation problems with a generous evaluation budget (e.g., 50-city-plus TSP), TS should not be assumed the safe default; tuning effort in SA's cooling schedule was this study's single largest source of apparent algorithm weakness. Across all three domains, PSO should be treated as requiring representation-matching before parameter-tuning.

5.7 Limitations

Results are drawn from one instance per domain and 10 runs per algorithm, consistent with this study's scope as a small demonstration (§1.3) rather than a full-scale experiment. The SA cooling-schedule correction means reported SA results reflect a tuned configuration rather than an untuned baseline; a systematic sensitivity sweep across cooling rates, tabu tenures, and swarm topologies was outside this study's budget. No formal significance testing beyond means and standard deviations was applied (§2.10, §2.11).

5.8 Conclusion

No single algorithm dominated across all three domains, consistent with the No Free Lunch principle (§2.1). Each algorithm's effectiveness instead tracked a specific, identifiable problem property: TS performed best where a small, well-matched local neighborhood existed or where the instance was small enough for any well-tuned method to reach the optimum; SA performed best where a long, budget-spanning search was needed to escape a neighborhood-limited local optimum; and PSO's performance tracked, almost linearly, how well its native representation matched the problem's underlying structure. This supports the study's original premise — that the value of comparing these three algorithms lies not in ranking them, but in mapping which problem properties predict which algorithm will perform well.

6. REFERENCES

[1] Alharkan, I., Saleh, M., Ghaleb, M. A., Kaid, H., Farhan, A., & Almarfadi, A. (2020). Tabu search and particle swarm optimization algorithms for two identical parallel machines scheduling problem with a single server. Journal of King Saud University–Engineering Sciences, 32(5), 330–338. https://doi.org/10.1016/j.jksues.2019.03.006

[2] Allvi, M. W., Hasan, M., Rayan, L., Shahabuddin, M., Khan, M. M., & Ibrahim, M. (2020). Feature selection for learning-to-rank using simulated annealing. International Journal of Advanced Computer Science and Applications, 11(3). https://doi.org/10.14569/IJACSA.2020.0110387

[3] Glover, F. (1989). Tabu search—Part I. ORSA Journal on Computing, 1(3), 190–206. https://doi.org/10.1287/ijoc.1.3.190

[4] Grabusts, P., Musatovs, J., & Golenkov, V. (2019). The application of simulated annealing method for optimal route detection between objects. Procedia Computer Science, 149, 95–101. https://doi.org/10.1016/j.procs.2019.01.112

[5] Huang, S., Tian, N., Wang, Y., & Ji, Z. (2016). Multi-objective flexible job-shop scheduling problem using modified discrete particle swarm optimization. SpringerPlus, 5, Article 1432. https://doi.org/10.1186/s40064-016-3054-z

[6] Jwo, J.-S., Lee, C.-H., Chen, J.-T., Lin, C.-S., Lin, C.-Y., Cheng, W.-K., Chang, C.-H., & King, J.-K. (2023). Application of tabu search for job shop scheduling based on manufacturing order swapping. Engineering Proceedings, 55(1), Article 51. https://doi.org/10.3390/engproc2023055051

[7] Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. In Proceedings of ICNN'95—International Conference on Neural Networks (Vol. 4, pp. 1942–1948). IEEE. https://doi.org/10.1109/ICNN.1995.488968

[8] Kirkpatrick, S., Gelatt, C. D., Jr., & Vecchi, M. P. (1983). Optimization by simulated annealing. Science, 220(4598), 671–680. https://doi.org/10.1126/science.220.4598.671

[9] Mhamdi, B., Grayaa, K., & Aguili, T. (2011). Hybrid of particle swarm optimization, simulated annealing and tabu search for the reconstruction of two-dimensional targets from laboratory-controlled data. Progress in Electromagnetics Research B, 28, 1–18. https://doi.org/10.2528/PIERB10112902

[10] Niño, E. D., Ardila, C. J., & Chinchilla, A. (2012). A novel, evolutionary, simulated annealing inspired algorithm for the multi-objective optimization of combinatorial problems. Procedia Computer Science, 9, 1992–1998. https://doi.org/10.1016/j.procs.2012.04.218

[11] Pirim, H., Bayraktar, E., & Eksioglu, B. (2008). Tabu search: A comparative study. In W. Jaziri (Ed.), Tabu search. IntechOpen. https://doi.org/10.5772/5637

[12] Ru, S. (2024). Vehicle logistics intermodal route optimization based on tabu search algorithm. Scientific Reports, 14, Article 11859. https://doi.org/10.1038/s41598-024-60361-7

[13] Sengupta, S., Basak, S., & Peters, R. A., II. (2019). Particle swarm optimization: A survey of historical and recent developments with hybridization perspectives. Machine Learning and Knowledge Extraction, 1(1), 157–191. https://doi.org/10.3390/make1010010

[14] Watson, J.-P., Whitley, L. D., & Howe, A. E. (2005). Linking search space structure, run-time dynamics, and problem difficulty: A step toward demystifying tabu search. Journal of Artificial Intelligence Research, 24, 221–261.

[15] Xie, H., Zhang, L., Lim, C. P., Yu, Y., & Liu, H. (2021). Feature selection using enhanced particle swarm optimisation for classification models. Sensors, 21(5), Article 1816. https://doi.org/10.3390/s21051816

[16] Youssef, H., Sait, S. M., & Adiche, H. (2001). Evolutionary algorithms, simulated annealing and tabu search: A comparative study. Engineering Applications of Artificial Intelligence, 14(2), 167–181. https://doi.org/10.1016/S0952-1976(00)00065-8

[17] Zhan, S.-H., Lin, J., Zhang, Z.-J., & Zhong, Y.-W. (2016). List-based simulated annealing algorithm for traveling salesman problem. Computational Intelligence and Neuroscience, 2016, Article 1712630. https://doi.org/10.1155/2016/1712630

[18] Zhang, H., & Sun, G. (2002). Feature selection using tabu search method. Pattern Recognition, 35(3), 701–711. https://doi.org/10.1016/S0031-3203(01)00046-2

[19] Zhang, W., & Nicholson, C. D. (2018). Empirical analysis of metaheuristic search techniques for the parameterized dynamic slope scaling procedure [Preprint]. arXiv. https://arxiv.org/abs/1808.10264

[20] Zhang, Y., Gong, D.-W., Sun, X.-Y., & Guo, Y.-N. (2017). A PSO-based multi-objective multi-label feature selection method in classification. Scientific Reports, 7, Article 376. https://doi.org/10.1038/s41598-017-00416-0